Quantumview - Interaktive Visualisierung hybrider Quantum Key Distribution

Florian Hesser
University of Applied Sciences Darmstadt
Darmstadt, Germany
florian.hesser@stud.h-da.de

Luca Stehle
University of Applied Sciences Darmstadt
Darmstadt, Germany
luca.stehle@stud.h-da.de

Aria Gholami
University of Applied Sciences Darmstadt
Darmstadt, Germany
aria.gholami@stud.h-da.de

Zusammenfassung

Diese Arbeit präsentiert eine webbasierte Lernanwendung zur anschaulichen Vermittlung von Quantum Key Distribution (QKD), QKD-Netzen und hybrider Schlüsseleinigung. Die Anwendung kombiniert einen verständnisorientierten Einstieg (Beginner-/Expert-Modus) mit einer interaktiven Visualisierung auf Basis von React Flow, die den Agile-Q-Key-Exchange mit Knoten, Kanten sowie der Rolle von KMS/HKS darstellt. Die Software wird als React-Anwendung über Nginx in einer Docker-Umgebung bereitgestellt; eine GitLab-basierte CI/CD-Pipeline ermöglicht reproduzierbare Builds und einen automatisierten Rollout. Sicherheitsrelevante Header, Zugangsschutz und eine produktionsnahe Local-Preview-Umgebung unterstützen einen sicheren Betrieb und die Qualitätssicherung. Damit schließt die Arbeit die Lücke zwischen protokollzentrierten Simulationen und der Einbettung der Schlüssel in einen System- und Netzwerkkontext.

ACM Reference Format:
Florian Hesser, Luca Stehle, and Aria Gholami. 2025. Quantumview - Interaktive Visualisierung hybrider Quantum Key Distribution. In *Proceedings of University of Applied Sciences Darmstadt, 2025 (Darmstadt, Deutschland).* , 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Einleitung

> „Historical studies of cryptography show that no matter how clever you are, and what kind of clever system you invent, sooner or later there will be someone, not even clever, who can break it“[2]
>
> — Artur Ekert, Erfinder des E91-Protokolls, Professor für Quantenphysik (Universität von Oxford)

Die fortschreitende Entwicklung von Quantumcomputern stellt eine ernstzunehmende Bedrohung für die Sicherheit klassischer Verfahren dar. Insbesondere asymmetrische Kryptosysteme wie RSA, die heute weit verbreitet in digitalen Kommunikations-und IT-Infrastrukturen eingesetzt werden, könnten durch leistungsfähige Quantumcomputer effizient gebrochen werden. Der SSH-Schlüssel mit dem RSA Algorithmus ist ein gutes Beispel, welches bis zur OpenSSH Version 8.8 (April 2022) als Default aktiviert war. [38] Im Rahmen eines Experiments an der Shanghai Universität wurde demonstriert, dass ein 22-Bit-RSA-Schlüssel mit einem Quantencomputer erfolgreich gebrochen werden kann. [7] [42] Moderne RSA-Schlüssel (bspw. für eine Schlüsselgenerierung mit OpenSSH) haben üblicherweise eine empfohlene Länge von 2048 Bit. [41] Somit bleibt der praktische Nutzen für reale Angriffe bislang aus, auch wenn das Experiment den grundsätzlichen Ansatz für so einen Angriff bestätigt. [7]

Vor diesem Hintergrund gewinnt Quantum Key Distribution (QKD) als quantensichere Methode zur symmetrischen Schlüsselverteilung zunehmend an Bedeutung. QKD ermöglicht es, Schlüssel zwischen 2 Parteien auszutauschen.

Um Kommunikationsnetze mit der QKD-Technologie über große Entfernungen zu realisieren, werden mehrere QKD-Module zu einem Netzwerk verbunden. [5] Dabei spielt ein Key Management System (KMS) eine zentralle Rolle, indem es die Schlüssel speichert und die sichere Schlüsselweiterleitung innerhalb des Netzwerks steuert. [21] Trotz der theoretisch hohen Sicherheit von QKD-Systemen bleiben praktische Implementierungen anfällig für diverse Sicherheitslücken und Angriffsvektoren.

Zur Erhöhung der Gesamtsicherheit kann auf hybride Schlüsselaustauschverfahren gesetzt werden, bei denen QKD mit Post-Quanten Kryptografie (PQC) Verfahren kombiniert werden. Ziel ist es, die Resilienz des Systems zu erhöhen, indem der resultierende Schlüssel weiterhin als sicher gilt, solange mindestens eines der zugrunde liegenden Verfahren nicht kompromittiert wurde. [5] Die Open-Source-Anwendung *Go-HybridizeKeyService* stellt in diesem Zusammenhang ein praktisches Werkzeug dar, um aus verschiedenen Schlüsselquellen einen robusten hybriden Schlüssel zu erzeugen.[18]

In Kapitel 2 stellen wir die für diese Arbeit relevanten Grundlagen zu QKD sowie zur Hybridisierung vor. Anschließend werden in Sektion 3 bestehende Arbeiten zu unserer Webanwendung betrachtet. Für die Umsetzung der Anwendung haben wir in Kapitel 4 die Methodik erläutert, wie wir uns

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Darmstadt, Deutschland , ,
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

zu dem Ergebnis hingearbeitet haben. Anschließend wird die Umsetzung der Applikation in Sektion 5 erklärt. Aus der Umsetzung haben wir in Kapitel 6 ein Ergebnis formuliert und darüberhinaus werden weitere Problematiken in Kapitel 7 diskutiert. Zum Schluss geben wir in Kapitel 8 ein Fazit über unser Projekt und in Kapitel 9 einen kurzen Ausblick über mögliche neue Features.

1.1 Motivation

Wie bereits in Abschnitt 1 (Einleitung) erwähnt, ist die praktische Umsetzung von QKD komplex, anfällig für Fehler und mit hohen Kosten verbunden. Aus diesem Grund wird die Hybridisierung eingesetzt, die als zusätzlicher Schutzmechanismus dient, um die Sicherheit gegenüber möglichen Schwachstellen zu erhöhen. [5] Allerdings macht die Hybridisierung das System insgesamt nicht leichter verständlich, sondern erschwert insbesondere Laien das Verständnis von QKD, obwohl sie für die Sicherheit von großer Bedeutung ist. [9] Die Motivation dieser wissenschaftlichen Arbeit liegt daher darin, einen Beitrag dazu zu leisten, dass auch fachfremde Personen die Funktionsweise und Bedeutung von QKD in Kombination mit Hybridisierung besser verstehen können.

1.2 Problemstellung

Trotz der in Abschnitt 1 dargelegten sicherheitstechnischen Vorteile der Quantum Key Distribution, bestehen weiterhin zentrale Herausforderungen, die einer breiten Implementierung im Wege stehen. Für die erfolgreiche Etablierung von QKD im praktischen Einsatz sind vorallem Vertrauen, gesellschaftliche Akzeptanz sowie eine ausreichende Investitionsbereitschaft erforderlich. Fehlt es an diesen Faktoren, wird sich die Technologie trotz ihres Potenzials nicht durchsetzen können. Ohne Vertrauen und Akzeptanz wird QKD kaum Anwendung finden und ohne finanzielle Mittel kann die notwendige, kostenintensive Infrastruktur nicht aufgebaut und betrieben werden. [9] [8]

Ein wesentliches Hindernis, das diese Voraussetzungen erschwert, ist die Komplexität der Technologie. Derzeit ist es kaum etabliert, insbesondere für Laien, die Funktionsweise von QKD gleichzeitig verständlich und fachlich korrekt zu vermitteln. Ein anschauliches Beispiel hierfür sind die bislang nur wenigen Arbeiten zur Integration von QKD in den Schulunterricht. Diese Beobachtung wurde unter anderem vom Jerusalem College of Technology bestätigt, das zudem darauf hinweist, dass QKD stärker in schulische Lehrpläne aufgenommen werden sollte. Diese Schwierigkeit wird durch aktuelle Entwicklungen, wie etwa die Integration der Hybridisierung in QKD zusätzlich verschärft. Dadurch entsteht ein Erklärungs-und Vermittlungsdefizit, das einer breiten gesellschaftlichen und wirtschaftlichen Akzeptanz im Weg steht. Dieses Problem stellt eine zentrale Herausforderung für die zukünftige Verbreitung und Nutzung von QKD dar. [44]

1.3 Zielsetzung

Ziel dieser Arbeit ist die Konzeption und Entwicklung einer Webanwendung, die die Funktionsweise von Quantum Key Distribution, sowie QKD Netzwerke verständlich vermittelt. Dabei sollen als Zielgruppe Personen mit informatischem oder betriebswirtschaftlichem Hintergrund gelten. Um unterschiedliche Ansätze zur Förderung des Wissens zu QKD bereitzustellen konzentriert sich diese Arbeit auf diverse Lernmethoden und bietet statt nur visuelle Einblicke mit Text, auch interaktive Einblicke in QKD wieder. Ein zentrales Anliegen besteht darin, langfristig beizutragen Akzeptanz und Verständnis für QKD zu fördern, sowie die Kommunikation zwischen technischen und nicht-technischen Stakeholdern zu erleichtern.

2 Grundlagen

Klassische asymmetrische Verfahren wie RSA, DH oder ECC werden von künftigen Quantencomputern bedroht. Um weiterhin sichere Kommunikation zu ermöglichen, verfolgt die Forschung aktuell zwei unterschiedliche Ansätze: Post-Quantum Cryptography (PQC) und Quantum Key Distribution (QKD). [5]

2.1 Post-Quantum Cryptography

Die klassichen Verfahren beruhen auf mathematischen Problem wie Faktorisierung oder dem diskreten Logarithmusproblem, die für klassische Computer als „schwer“ gelten. Ein Quantencomputer könnte diese Probleme durch Nutzung des Shor-Algorithmus jedoch in polynomialer Zeit lösen und damit die bestehen Verfahren brechen. Der Begriff PQC bündelt daher alle Krypto-Algorithmen, deren Sicherheit auf mathematischen Problemen basiert, von denen angenommen wird, dass sie auch für Quantuencomputer schwer zu lösen sind. [25] Die US-Standardisierungsbehörde NIST hat 2024 die ersten drei Federal Information Processing Standards (FIPS)-Standards (FIPS 203 Kyber/ML-KEM, FIPS 204 Dilithium/ML-DSA, FIPS 205 Sphincs+/SLH-DSA) finalisiert und empfiehlt Unternehmen, mit der Migration zu beginnen [32]. Die drei im August 2024 verabschiedeten PQC-Standards sind folgende:

- FIPS 203 - Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM): ML-KEM geht aus dem CRYSTALS-Kyber-Beitrag hervor und definiert einen Key-Encapsulation Mechanism (KEM), mit dem zwei Kommunikationspartner über einen öffentlichen Kanal einen gemeinsamen geheimen Schlüssel aushandeln können. Damit ergänzt FIPS 203 die bisher zulässigen Schlüsselaustauschverfahren aus SP 800-56A (diskrete-Logarithmusbasierte Verfahren, z. B. Diffie-Hellman/ECC) und SP 800-56B (Faktorisierungs-basierte Verfahren, z. B. RSA). [31]
- FIPS 204 – Module-Lattice-Based Digital Signature Standard (ML-DSA): Dieser Standard ist aus dem CRYSTALS-Dilithium-Vorschlag abgeleitet und spezifiziert ein gitterbasiertes Signaturverfahren für Authentizität und Integrität digitaler Daten.[31]

- FIPS 205 – Stateless Hash-Based Digital Signature Standard (SLH-DSA): Basierend auf SPHINCS+ beschreibt FIPS 205 ein zustandsloses Hash-basiertes Signaturverfahren, das ohne private Schlüsselverwaltung auskommt und damit langfristige Vertrauensanker erleichtert. [31]

Allerdings zeigt eine Capgemini Studie von 2025, dass lediglich 15 % der Unternehmen einen PQC-Ansatz implementiert haben, was laut den Autoren ein deutliches Defizit darstellt [6].

2.2 Quantum Key Distribution

Quantum Key Distribution (QKD) ist ein Verfahren, welches den Quanteneffekt No-Cloning-Theorem nutzt, um einen Schlüssel zwischen zwei Kommunikationspartnern auszutauschen. Dabei verbinden sich Kommunikationspartner über einen Quantenkanal und einen klassischen authentifizierten, aber nicht verschlüsselten Kanal. [40]. Im BB84-Protokoll polarisiert Alice einzelne Photonen in zufälligen Polarisationsbasen und schickt diese polarisiteren Photonen an Bob. Bob misst in ebenfalls zufällig gewählten Basen, wodurch er entweder die richtige oder eben die falsche Basis wählt. Stimmen die Basen überein, entsteht ein gemeinsamer Schlüssel, nachdem über einen öffentlichen Kanal die Basenwahl geteilt wurde. Durch das No-Cloning-Theorem kann eine hohe Fehlerrate darauf hinweisen, dass ein Man-in-the-Middle-Angriff stattgefunden hat und der Schlüssel verworfen werden kann. [3]

In den vergangenen Jahren hat die Forschung QKD aus dem Labor in reale Netze überführt. Ein aktuelles Beispiel dafür ist das von Januar 2022 bis Dezember 2024 durchgeführte Projekt DemoQuanDT, in welchen eine quantensichere Verbindung über 923 km zwischen Berlin und Bonn mit 15 Trusted Nodes realisiert wurde. [19] Somit zeigt dieses Projekt, dass sich schon heutzutage QKD in bestehende Netzinfrastrukturen integrieren lassen.

2.3 Hybridisierung

PQC-Verfahren gelten zwar für künftige Quantencomputer als mathematisch schwer zu lösen, allerdings sind diese Algorithmen vorallem gegen Seitenkanalresitenz und Implementierungssicherheit deutlich kürzer erprobt als die klassischen Algorithmen. Das BSI empfiehlt deshalb, PQC nicht isoliert, sondern hybrid in Kombination mit einem klassischen Verfahren einzusetzen. Somit findet zunächst ein Schlüsselaustausch über RSA, ECDH oder anderen klassischen Protokollen und anschließend ein zweiter, quantensicherer Austausch über beispielsweise Kyber (FIPS 203). Die beiden resultierenden Shared-Secrets werden mit einer Key-Derivation-Function zu einem Hybridschlüssel verknüpft. [5] Für Anwendungen mit besonders hohen Vertraulichkeitsanforderungen kann zusätzlich ein QKD-Schlüssel als drittes Entropieelement in die KDF eingebracht werden. Der beschriebene Ablauf wird in Abbildung 1 schematisch dargestellt. Dabei zeigen aktuelle Studien, dass TLS sich erweitern lassen, ohne den Verbindungsaufbau spürbar zu verzögern [4].

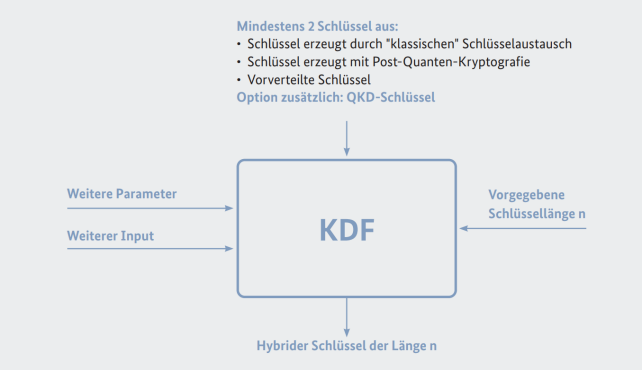


Abbildung 1: Schematische Darstellung der hybriden Schlüsseleinigung mittels einer Key Derivation Function (KDF) [5]

3 Verwandte Arbeiten

Es gibt bereits mehrere frei zugängliche Anwendungen, die den Ablauf des BB84-Protokolls und den Aufbau von QKD-Netzen anschaulich vermitteln.

QuVis – „BB84 Photons“. Die QuVis-Simulation präsentiert Polarisation und Basiswahl in einer spielerischen Animation, sodass Lernende das Protokoll unmittelbar „hands-on“ erkunden können. Die textuelle Einführung ist bewusst knapp gehalten, stattdessen setzt das Tool auf einen *Learning-by-Doing*-Ansatz. Neben dem regulären Schlüsselaustausch lässt sich ein *Man-in-the-Middle*-Szenario einschalten, um zu demonstrieren, wie ein erhöhter Quantum Bit Error Rate (QBER) einen Lauschangriff entlarvt. Der Funktionsumfang bleibt jedoch auf den reinen Ablauf von BB84 beschränkt. Weiterführende Themen wie Schlüsselverwendung, Hybridisierung oder Systemintegration werden nicht behandelt. [39]

„BB84 QKD Simulator“ (Streamlit). Auch diese Web-App konzentriert sich ausschließlich auf das Kernprotokoll. Im Gegensatz zu QuVis verzichtet sie auf Animationen und legt den Schwerpunkt auf die numerische Auswertung des QBER, um festzustellen, ob der Schlüsselaustausch kompromittiert wurde. Inhalte über den Kontext oder die Weiterverwendung des Schlüssels fehlen hier ebenfalls. [24]

OpenQKD-Dashboard. Einen anderen Fokus setzt das EU-Projekt *OpenQKD*. Das webbasierte Dashboard erlaubt den Aufbau virtueller QKD-Netzwerke und analysiert dabei Metriken wie Schlüsselrate, Link-Status und Topologie. Obwohl die Plattform damit wertvolle Einblicke in den Betrieb großskaliger QKD-Infrastrukturen bietet, adressiert sie weder den Einstieg in BB84 noch die Kombination der erzeugten QKD-Schlüssel mit quantenresistenten (PQC-)Verfahren. [37]

Zusammenfassend illustrieren alle drei Werkzeuge wichtige Teilaspekte von BB84 beziehungsweise QKD-Netzen, decken jedoch nicht den gesamten Bedarf ab, insbesondere wenn es um die Einbettung der Schlüssel in hybride, quantum sichere Systeme geht.

4 Methodik

Ausgehend von den in Abschnitt 1 skizzierten Herausforderungen sowie der Analyse verwandter Arbeiten (Abschnitt 3) wurde eine iterative und inkrementelle Vorgehensweise gewählt. Ziel war die nachvollziehbare, reproduzierbare Entwicklung einer interaktiven Webanwendung, die QKD, QKD-Netze und deren Hybridisierung verständlich vermittelt und dabei technisch korrekt bleibt.

4.1 Anforderungsableitung und Gestaltungsprinzipien

Die in Abschnitt 1 (Motivation und Problemstellung) und Abschnitt 2 dargelegten Inhalte bildeten die Grundlage für die Ableitung der funktionalen und didaktischen Anforderungen. Im Fokus standen:

- verständliche Vermittlung von QKD-Abläufen sowie QKD-Netzwerken,
- Visualisierung der Rolle von KMS/HKS und der Hybridisierung (Schlüsselkombination),
- interaktive Elemente zur schrittweisen Erkundung (Navigation durch einzelne Schritte),
- technische Korrektheit bei gleichzeitig reduzierter Komplexität für die Zielgruppe.

4.2 Technologie- und Architekturentscheidungen

Für die Umsetzung wurde eine komponentenbasierte Webarchitektur gewählt (vgl. Abschnitt 5.1). Das Frontend basiert auf React; die Bereitstellung erfolgt über einen Nginx-Webserver innerhalb einer Docker-Umgebung. Sicherheitsrelevante Servereinstellungen (u. a. Zugangsschutz, CORS-Header, Clickjacking- und MIME-Sniffing-Schutz) wurden für den Produktivbetrieb aktiviert. Für die interaktive Visualisierung der Prozesse (Agile-Q-Key Exchange) wurde die Bibliothek *React Flow* eingesetzt, um Knoten (Systemkomponenten) und Kanten (Relationen) dynamisch darzustellen (vgl. Abschnitt „React-Flow“).

4.3 CI/CD, Auslieferung und Preview

Zur Sicherstellung reproduzierbarer Builds und einer automatisierten Auslieferung wurde eine CI/CD-Pipeline in GitLab etabliert (vgl. Abschnitt 5.2). Das Image wird via Docker-in-Docker gebaut und in einer GitLab Registry bereitgestellt. Die Auslieferung auf das Zielsystem erfolgt idempotent mittels Ansible-Playbooks; der finale Rollout wird durch ein Shell-Skript angestoßen. Für lokale, produktionsnahe Tests wurde eine „Local Preview“-Umgebung mit Traefik und selbstsignierten Zertifikaten eingesetzt (vgl. Abschnitt „Local Preview“). Aufgrund fehlender DNS-Einträge erfolgt der aktuelle Produktivbetrieb ohne HTTPS, die relevanten Sicherheitsheader und ein Zugangsschutz sind jedoch aktiv.

4.4 Qualität, Sicherheit und Nachvollziehbarkeit

Qualität und Sicherheit wurden entlang der folgenden Maßnahmen adressiert:

- **Reproduzierbarkeit**: Containerisierte Builds, Versionierung über Git, automatisierte Pipeline-Schritte.
- **Sichere Bereitstellung**: Härtung des Webservers (Header, Zugangsschutz, Logging) für die Auslieferung.
- **Robuste Visualisierung**: Begrenzung von Zoom und Drag in der React-Flow-Ansicht, um eine konsistente Darstellung und Nutzerführung sicherzustellen.

4.5 Abgrenzung

Die Anwendung fokussiert auf das Verstehen zentraler Konzepte (QKD, KMS/HKS, Hybridisierung) durch interaktive Visualisierung. Eine vollständige Abbildung realer QKD-Implementierungen oder eine Integration externer Schlüsselquellen über Backend-Schnittstellen ist nicht Gegenstand dieser Arbeit. Die Hybridisierung wird konzeptionell und visuell (u. a. „Key Combination“) dargestellt; Details zur kryptographischen Schlüsselableitung werden im Kontext der Grundlagen (Abschnitt 2) und der Umsetzung behandelt.

5 Umsetzung

5.1 Komponenten

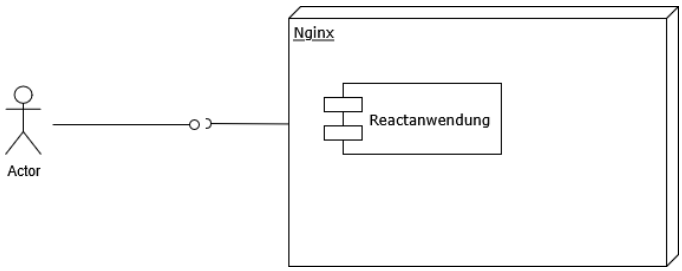


Abbildung 2: Komponentendiagramm zur Webanwendung

Abbildung 2 zeigt die grundlegende Architektur der entwickelten Webanwendung. Es handelt sich dabei um eine React-Anwendung, die über einen Nginx-Webserver bereitgestellt wird. Der Server hört auf Port 80 (HTTP) und ist über die entsprechende IP-Adresse und Portangabe im Browser erreichbar. [15] [35]

Für den produktiven Betrieb des Nginx-Servers wurden mehrere sicherheitsrelevante Einstellungen vorgenommen: [28] [33] [34] [30] [29]

- Aktivierung von CORS-Headern zur Steuerung von Cross-Origin-Anfragen,
- Passwortbasierte Authentifizierung (Zugangsschutz),
- Aktiviertes Access Logging zur Nachvollziehbarkeit von Zugriffen,
- Schutz vor Clickjacking durch Setzen des Headers `X-Frame-Options: DENY`,
- Verhinderung der automatischen MIME-Typ-Erkennung mittels `X-Content-Type-Options: nosniff`.

Der Nginx-Server läuft in einer Docker-Compose-Umgebung, die das benötigte Image aus der GitLab Registry der Hochschule Darmstadt bezieht. [16] Innerhalb des Containers wird

Port 80 freigegeben. [11] 2 zusätzliche Dateien werden über Volume-Mounts in das Dateisystem des Containers eingebunden: [12]

(1) Eine Datei mit dem Base64-codierten Passwort und Benutzernamen für die Authentifizierung,
(2) Die angepasste nginx-Konfigurationsdatei.

Das Image wird auf Basis eines eigenen Dockerfiles gebaut. [10] Darin wird zunächst die React-Anwendung mit Node.js Version 22 (derzeitig LTS Version [36]) im Produktionsmodus kompiliert und geminified. Dieser Schritt ist erforderlich, damit bspw. die V8 JavaScript Engine im Browser den Code effizient in Maschinensprache übersetzen und ausführen kann. [43] Anschließend werden die generierten statischen Dateien in das Verzeichnis überführt, das vom Nginx-Server an den Besucher ausgeliefert wird. [35]

Die Entscheidung für React als Frontend Framework wurde aufgrund der geplanten langfristigen Ausrichtung auf Interaktivität getroffen. React bietet durch seine komponentenbasierte Architektur und die Nutzung der Hooks API eine flexible und effiziente Möglichkeit zur Entwicklung dynamischer Benutzeroberflächen. Dies ist insbesondere im Kontext einer webbasierten Lernanwendung von Vorteil, da Inhalte modular aufgebaut, leicht angepasst und interaktiv gestaltet werden können. [14]

5.2 Auslieferung

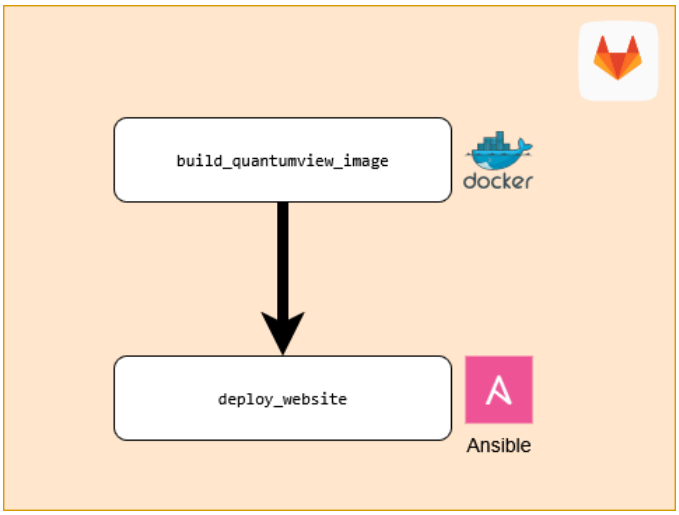


Abbildung 3: GitLab CI/CD Pipeline

Die Grafik 3 veranschaulicht den Aufbau der eingesetzten CI/CD-Pipeline. Diese gliedert sich in zwei wesentliche Phasen: den Build-Prozess der Anwendung sowie deren anschließende Auslieferung auf das Zielsystem.

Der Build-Prozess erfolgt unter Verwendung von Docker-in-Docker (DinD). Hierbei wird innerhalb der GitLab CI-Umgebung ein Image basierend auf einem bereitgestellten Dockerfile erzeugt. Diese wird anschließend in einer GitLab Container Registry abgelegt. Docker fungiert in diesem Kontext als zentrales Werkzeug zur Erstellung und Verwaltung containerisierter Applikationen. [17]

Die Auslieferung der Anwendung erfolgt automatisiert mittels Ansible. Ansible ist ein deklaratives Konfigurationsmanagement-Werkzeug, das es ermöglicht, wiederholbare und nachvollziehbare Aufgaben serverseitig auszuführen. Im Rahmen dieser Arbeit wurden zwei zentrale Ansible-Playbooks implementiert: Eines zur Grundkonfiguration des Zielsystems (z.B. Installation von Docker) sowie ein weiteres zur Bereitstellung und Aktualisierung der Webanwendung. [1]

Ein wesentlicher Vorteil von Ansible besteht in der idempotenten Ausführung seiner Tasks. So wird beispielsweise bei der Paketinstallation geprüft, ob ein Paket bereits vorhanden ist, um redundante Installationen zu vermeiden. Dies erhöht die Effizienz und trägt zu einer konsistenten Systemkonfiguration bei. Gleichzeitig dient die Playbook-Struktur als dokumentierte Grundlage für den Systemzustand. [1]

Die finale Auslieferung der Anwendung wird durch das Skript *.deploy/roles/rollout-release/files/rollout-release.sh* angestoßen. Dieses Skript startet die produktive Docker-Compose-Umgebung mit der jeweils neusten Image-Version der Webanwendung. [13]

5.3 Local Preview

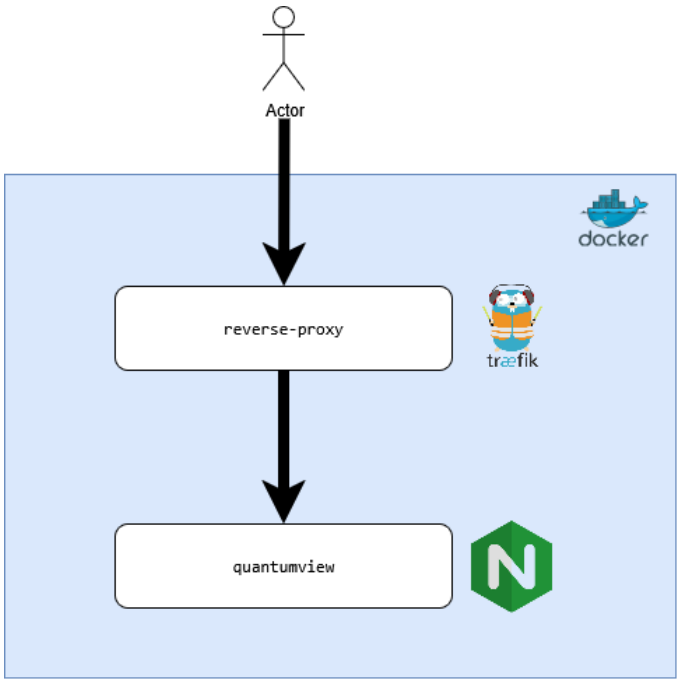


Abbildung 4: Local Preview Umgebung

Die Local Preview Umgebung stellt eine produktiv-nahe Testumgebung dar, in der neue Features vor der Auslieferung getestet werden können. Die Darstellung 4 zeigt die grundlegende Architektur dieser Umgebung. Diese besteht aus zwei Docker-Containern:

(1) Container mit einem Traefik-Image
(2) Container mit dem gebauten Image aus der GitLab Registry

Der Bau der Webanwendung bzw. des Images wurde bereits in Abschnitt 5.2 erläutert.

Traefik ist speziell für containerisierte Anwendungen konzipiert und ermöglicht unter anderem die automatische Konfiguration von HTTPS-Verbindungen mithilfe lokal generierter, selbstsignierter Zertifikate. [26] [27] Die Namensauflösung (DNS) erfolgt hierbei lokal, sodass keine Zertifikate einer offiziellen Zertifizierungsstelle verwendet werden. [20] Entsprechend gelten die Zertifikate standardmäßig nicht als vertrauenswürdig durch Browser. Dennoch erlaubt diese Konfiguration eine realitätsnahe Testumgebung, die zentrale Aspekte der späteren Auslieferungsszenarien abbildet. [22]

In der aktuellen Produktivumgebung wird weder ein Reverse Proxy eingesetzt noch erfolgt die Bereitstellung des Webanwendung über HTTPS. Der Verzicht auf eine SSL-Verschlüsselung begründet sich dadurch, dass für die zugrunde liegende IP-Adresse kein DNS-Eintrag existiert. Dies verhindert eine Zertifikatsausstellung über Let's Encrypt, da diese derzeit ausschließlich für gültige Domainnamen unterstützt wird. Zwar befindet sich die Unterstützung für IP-basierte Zertifikate aktuell in der Staging-Phase von Let's Encrypt, ist jedoch bislang nicht produktiv verfügbar. [23] Darüber hinaus wird die Nutzung von Let's Encrypt Zertifikaten ohne gültige Domainbindung, beispielsweise lokale Hostnamen wie „localhost", von Let's Encrypt nicht empfohlen [22]. Aus diesem Grund wird in der vorliegenden Umgebung ein selbstsigniertes Zertifikat verwendet, das nicht von einer offiziellen Zertifizierungsstelle ausgestellt wurde. Darüberhinaus wurde auf die Umsetzung sicherheitsrelevanter Webserverkonfigurationen in der Local Preview verzichtet (vgl. Abschnitt 5.1).

5.4 React-Flow

Für die Implementierung der Agile-Q-Key Exchange-Grafik wurde die JavaScript-Bibliothek React Flow verwendet. Diese ermöglicht die dynamische und responsive Darstellung von Graphstrukturen innerhalb webbasierter Anwendungen. Die zugrunde liegende Struktur der Grafik orientiert sich an dem Konzept von Knoten (Nodes) und Kanten (Edges), wie es auch aus Graphdatenbanken bekannt ist. In diesem Kontext repräsentieren die Nodes einzelne Systemkomponenten, während die Edges die zwischen ihnen bestehenden Relationen visualisieren. Die in React Flow verwendeten Nodes verfügen über eine standardisierte Darstellung, können jedoch mithilfe von benutzerdefinierten Types flexibel erweitert werden. [45] Abbildung 5 veranschaulicht den Ablauf der Agile-Q-Key Exchange-Prozesse und zeigt, welche Schritte zu welchen Zeitpunkten durchlaufen werden.

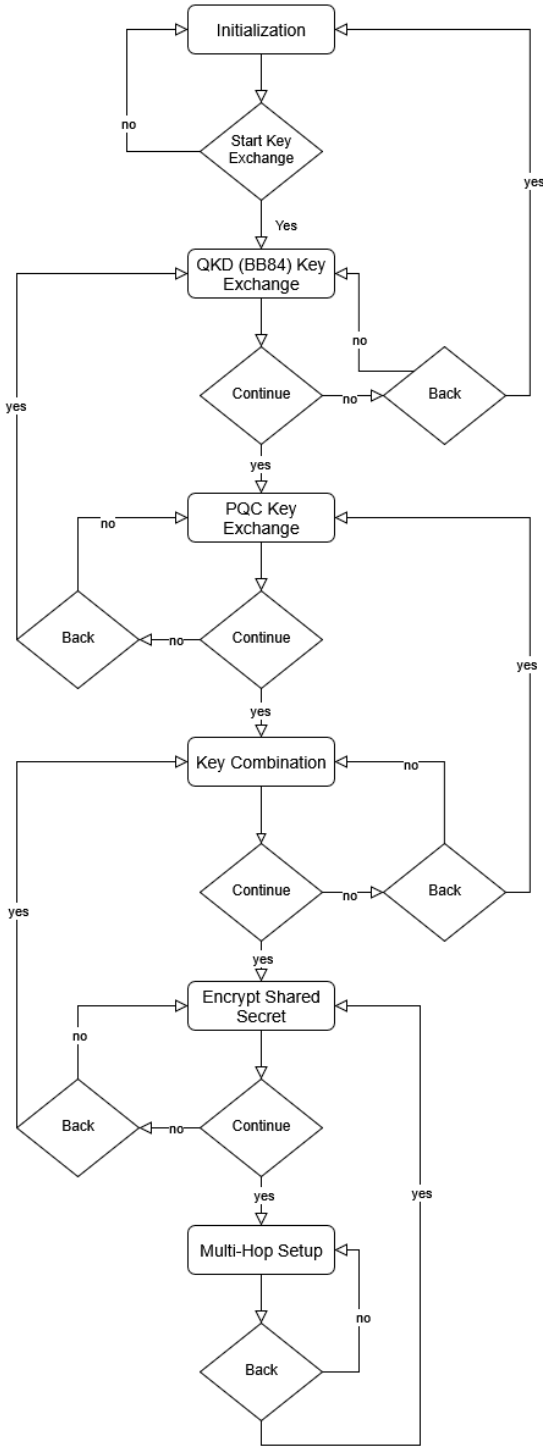


Abbildung 5: Ablaufdiagramm für Agile-Q-Key Exchange Grafik

6 Ergebnisse

Im Rahmen des Projekts wurde eine funktionsfähige Webanwendung entwickelt, die die Funktionsweise von Quantum Key Distribution (QKD) und deren Einbettung in hybride Schlüsselverfahren anschaulich darstellt. Zentrale Ergebnisse sind:

- **Interaktive Visualisierung**: Eine React-Flow-basierte Ansicht stellt den „Agile Q Key Exchange“ mit Knoten (z. B. Node A/B/C) und Kanten dar. KMS/HKS, PQC-Key-Module und QKD-Module werden sichtbar gemacht. Schrittweise Navigation („Back“, „Continue“) und ein „More information“-Link unterstützen das Verständnis; u. a. wird „Step 5: Forwarding in the Network“ dargestellt (Trusted Nodes).
- **Didaktische Startseite**: Die Anwendung bietet einen „Beginner Mode“ und einen „Expert Mode“. Im Beginner-Modus werden u. a. „What is Quantum Key Distribution?“ und „Why is Quantum Key Distribution important?“ erläutert.
- **Stabile Bedienung der Grafik**: Zoom-Steuerung ist vorhanden (u. a. minimale Vergrößerung), Knoten sind nicht verschiebbar; Wasserzeichen der Bibliothek wurden entfernt, um Lesbarkeit zu erhöhen.
- **Bereitstellung und Sicherheit**: Auslieferung über Nginx in Docker; aktivierte Sicherheitsheader (u. a. CORS, X-Frame-Options, X-Content-Type-Options) sowie Zugangsschutz und Logging für den Produktivbetrieb.
- **CI/CD und Reproduzierbarkeit**: Automatisierter Build (Docker-in-Docker) mit Ablage in einer Registry und automatisierter Rollout mittels Ansible; Local-Preview-Umgebung mit Traefik und selbstsigniertem Zertifikat.

7 Diskussion

Didaktische Einordnung. Die Kombination aus zweistufigem Einstiegsbereich (Beginner/Expert) und einer schrittweisen, interaktiven Visualisierung adressiert die in Abschnitt 1 benannten Hürden der Verständlichkeit. Die Einbettung von KMS/HKS und die Darstellung der Hybridisierung (u. a. „Key Combination“) schließen die in Abschnitt 3 identifizierte Lücke zwischen Protokoll-Visualisierungen und Systemkontext.

Technische Einordnung. Die containerisierte Bereitstellung mit Nginx sowie die automatisierte Auslieferung über CI/CD erhöhen Nachvollziehbarkeit und Reproduzierbarkeit der Ergebnisse (vgl. Abschnitt 5.2). Sicherheitsrelevante Header und Zugangsschutz sind für den Produktivbetrieb aktiviert. Die Local-Preview-Umgebung bietet eine realitätsnahe Testmöglichkeit, verzichtet jedoch bewusst auf vollständige Härtung.

Grenzen. Wie in Abschnitt 4 (Abgrenzung) beschrieben, fokussiert die Anwendung auf die konzeptionelle Vermittlung. Eine tiefe Integration realer Schlüsselquellen oder eine vollständige Abbildung produktiver QKD-Infrastrukturen ist nicht Bestandteil dieser Arbeit. In der aktuellen Produktivumgebung fehlt aufgrund ausstehender DNS-Konfiguration eine HTTPS-Bereitstellung; in der Preview werden selbstsignierte Zertifikate genutzt (vgl. Abschnitt „Local Preview“).

8 Fazit

Die entwickelte Anwendung erfüllt die Zielsetzung, QKD, QKD-Netze und deren Hybridisierung adressatengerecht zu vermitteln. Durch eine klar strukturierte Benutzerführung (Beginner/Expert), eine interaktive, schrittweise Visualisierung und eine reproduzierbare, sicherheitsbewusste Bereitstellung wird der Zugang zu den zentralen Konzepten erleichtert. Die Arbeit leistet damit einen Beitrag zur Verständlichkeit von QKD im Systemkontext.

9 Ausblick

Vor dem Hintergrund der beschriebenen Grenzen und der bestehenden Architektur ergeben sich folgende, naheliegende Weiterentwicklungen:

- **HTTPS-Produktivbetrieb**: Einrichtung eines Reverse Proxys und Ausstellung vertrauenswürdiger Zertifikate, sobald ein DNS-Name verfügbar ist (vgl. Abschnitt „Local Preview“).
- **Erweiterung der Inhalte**: Vertiefung des „Expert Mode“ und Ausbau der Schritttexte/Infotexte in der interaktiven Ansicht.
- **Systemintegration**: Optionale Anbindung externer Schlüsselquellen zur Demonstration der Hybridisierung im End-to-End-Fluss (im Rahmen der Abgrenzung bislang nicht umgesetzt; vgl. Abschnitt 2).
- **Spielerische Interaktionen**: Integration von interaktiven, spielerischen Elementen direkt im QKD-/Agile-Q-Key-Exchange-Diagramm zur Steigerung der Motivation.
- **Austausch und Diskussion**: Forum bzw. Kommentarbereich zur Unterstützung des inhaltlichen Austauschs.
- **FAQ-Bereich**: Häufige Fragen rund um QKD und Kryptographie bündeln und verständlich beantworten.
- **UX und Internationalisierung**: Dark Mode, allgemeine UX-Optimierungen und optionale Sprachauswahl.
- **Mobiloptimierung**: Weitere Verbesserungen für mobile Endgeräte,
  - Alternative Ansicht mit vereinfachter Bedienung und optimierter Darstellung.
- **Usability-Tests**: Gezielte Tests mit Nutzerinnen und Nutzern zur iterativen Verbesserung von Bedienbarkeit und Verständnis.
- **Ressourcenverweise**: Relevante Literatur und weiterführende Quellen innerhalb der Anwendung verlinken.
- **Interaktives Glossar**: Kontextbezogene Tooltips und ein Glossar für Fachbegriffe (z. B. QBER, KMS/HKS, Trusted Node).
- **Barrierefreiheit**: Verbesserungen hinsichtlich Tastaturnavigation und Kontrast (Ausrichtung an WCAG-Empfehlungen).
- **Performance**: Code-Splitting und Lazy Loading für Diagramme und Inhalte, um Ladezeiten weiter zu reduzieren.

9.1 Verweis auf Abbildungen im Anhang

Zur Illustration der Ergebnisse verweisen wir auf zwei Screenshots im Anhang: (i) die Startseite mit „Beginner Mode“ und „Expert Mode“ sowie (ii) die Visualisierung „Agile Q Key

Exchange“ mit „Step 5: Forwarding in the Network“. Die Abbildungen sind in Anhang A referenziert.

Literatur

[1] Ansible. 2025. *Introduction to Ansible.* https://docs.ansible.com/ansible/latest/getting_started/introduction.html

[2] NCCR Automation. [n. d.]. *“You can’t cheat physics”: Artur Ekert explains why only quantum cryptography can ensure security.* https://nccr-automation.ch/news/2025/you-cant-cheat-physics-artur-ekert-explains-why-only-quantum-cryptography-can-ensure

[3] Charles H. Bennett and Gilles Brassard. 2014. Quantum cryptography: Public key distribution and coin tossing. *Theoretical Computer Science* 560 (2014), 7–11. https://doi.org/10.1016/j.tcs.2014.05.025 Theoretical Aspects of Quantum Cryptography – celebrating 30 years of BB84.

[4] Javier Blanco-Romero, Pedro Otero García, Daniel Sobral-Blanco, Florina Almenares Mendoza, Ana Fernández Vilas, and Rebeca P. Díaz-Redondo. 2025. QKD-KEM: Hybrid QKD Integration into TLS with OpenSSL Providers. arXiv:2503.07196 [cs.CR] https://arxiv.org/abs/2503.07196

[5] Bundesamt für Sicherheit in der Informationstechnik. 2021. *Kryptografie quantensicher gestalten – Grundlagen, Entwicklungen, Empfehlungen.* Leitfaden. Bundesamt für Sicherheit in der Informationstechnik (BSI), Bonn. https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Publikationen/Broschueren/Kryptografie-quantensicher-gestalten.pdf

[6] Capgemini Research Institute. 2025. *Future Encrypted: Why Post-Quantum Cryptography Tops the New Cybersecurity Agenda.* White Paper. Capgemini, Paris. https://www.capgemini.com/wp-content/uploads/2025/07/Final-Web-Version-Report-PQC.pdf

[7] WANG Chao, WANG Qi Di, HONG Chun Lei, HU Qiao Yun, and PEI Zhi. 2021. Quantum Annealing Public Key Cryptographic Attack Algorithm Based on D-Wave Advantage. *Chinese Journal of Computers* 47 (Mai 2021), 1–2. https://doi.org/10.11897/SP.J.1016.2024.01030

[8] Technische Universität Darmstadt. [n. d.]. *Sichere Kommunikation mit Quantenschlüsseln.* https://www.tu-darmstadt.de/information-and-intelligence/researchfield_ii/news_events_ii/newsdetails_ii_497856.de.jsp?utm_source=chatgpt.com

[9] Eleni Diamanti, Hoi-Kwong Lo, Bing Qi, and Zhiliang Yuan. 2016. Practical challenges in quantum key distribution. *npj Quantum Information 2,16025 (2016)* 2 (Juni 2016), 1–14. https://arxiv.org/pdf/1606.05853

[10] Docker. 2025. *Multi-stage builds.* https://docs.docker.com/build/building/multi-stage/

[11] Docker. 2025. *Publishing and exposing ports.* https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/

[12] Docker. 2025. *Volumes.* https://docs.docker.com/engine/storage/volumes/

[13] Docker Docs. 2025. *docker compose.* https://docs.docker.com/reference/cli/docker/compose/

[14] Facebook. 2025. *Quick Start.* https://react.dev/learn

[15] geeksforgeeks. 2025. *What is Port 80?* https://www.geeksforgeeks.org/computer-networks/what-is-port-80/

[16] GitLab. 2025. *GitLab container registry.* https://docs.gitlab.com/user/packages/container_registry/

[17] GitLab. 2025. *Use Docker to build Docker images.* https://docs.gitlab.com/ci/docker/using_docker_build/

[18] Hochschule Darmstadt. [n. d.]. . https://code.fbi.h-da.de/hybridizekeyservice/gohybridizekeyservice

[19] P. Horoschenkoff, J. Henrich, R. Böhn, I. Khan, J. Rödiger, M. Gunkel, M. Bauch, J. Benda, P. Bläcker, E. Eichhammer, U. Eismann, G. Frenck, H. Griesser, W. Jontofsohn, N. Kopshoff, S. Röhrich, F. Seidl, N. Schark, E. Sollner, D. von Blanckenburg, A. Heinemann, M. Stiemerling, and M. Gärtner. 2025. DemoQuanDT: A Carrier-Grade QKD Network. arXiv:2503.21186 [quant-ph] https://arxiv.org/abs/2503.21186

[20] IONOS. 2025. *Was ist die hosts-Datei?* https://www.ionos.de/digitalguide/server/konfiguration/hosts-datei/

[21] Industry Specification Group (ISG). [n. d.]. *Definitions and Abbreviations.* https://www.etsi.org/deliver/etsi_gs/QKD/001_099/004/01.01.01_60/gs_QKD004v010101p.pdf

[22] Internet Security Research Group (ISRG). [n. d.]. *Certificates for localhost.* https://letsencrypt.org/docs/certificates-for-localhost/

[23] Internet Security Research Group (ISRG). [n. d.]. *How To Get an IP Address Cert.* https://letsencrypt.org/2025/07/01/issuing-our-first-ip-address-certificate

[24] Ruchir Kalokhe. 2025. *BB84 Quantum Key Distribution Simulator.* https://qkd-sim.streamlit.app/ Interactive Streamlit app; source code on GitHub.

[25] Manoj Kumar and Pratap Pattnaik. 2020. Post-Quantum Cryptography (PQC) – An Overview. In *2020 IEEE High Performance Extreme Computing Conference (HPEC).* IEEE, 1–9. https://doi.org/10.1109/HPEC43674.2020.9286140

[26] Traefik Labs. 2025. *Getting Started with Docker and Traefik.* https://doc.traefik.io/traefik/getting-started/docker/

[27] Traefik Labs. 2025. *TLS.* https://doc.traefik.io/traefik/https/tls/

[28] Mozilla. 2025. *Cross-Origin Resource Sharing (CORS).* https://developer.mozilla.org/de/docs/Web/HTTP/Guides/CORS

[29] Mozilla. 2025. *X-Content-Type-Options header.* https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Content-Type-Options

[30] Mozilla. 2025. *X-Frame-Options header.* https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options

[31] National Institute of Standards and Technology. 2024. Announcing Approval of Three Federal Information Processing Standards (FIPS) for Post-Quantum Cryptography. https://csrc.nist.gov/news/2024/postquantum-cryptography-fips-approved. CSRC news update, accessed 2025-07-11.

[32] National Institute of Standards and Technology. 2024. NIST Releases First 3 Finalized Post-Quantum Encryption Standards. https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards. News release, accessed 2025-07-11.

[33] Nginx. 2025. *Module ngx_http_auth_basic_module.* https://nginx.org/en/docs/http/ngx_http_auth_basic_module.html

[34] Nginx. 2025. *Module ngx_http_log_module.* https://nginx.org/en/docs/http/ngx_http_log_module.html#access_log

[35] Nginx. 2025. *Serving Static Content.* https://nginx.org/en/docs/beginners_guide.html

[36] Node.js. 2025. *Node.js Releases.* https://nodejs.org/en/about/previous-releases

[37] OPENQKD Consortium. 2021. *OPENQKD– Open European Quantum Key Distribution Testbed.* https://open-qkd.eu/ Projektwebsite, EU Horizon2020.

[38] OpenSSH. 2021-09-26. *Potentially-incompatible changes.* https://www.openssh.com/txt/release-8.8

[39] QuVis Project, School of Physics and Astronomy, University of St Andrews. [n. d.]. *Quantum Key Distribution (BB84) with Polarized Photons — Interactive Simulation.* https://www.st-andrews.ac.uk/physics/quvis/simulations_html5/sims/BB84_photons/BB84_photons.html Educational simulation for teaching quantum cryptography.

[40] Valerio Scarani, Helle Bechmann-Pasquinucci, Nicolas J. Cerf, Miloslav Dušek, Norbert Lütkenhaus, and Momtchil Peev. 2009. The security of practical quantum key distribution. *Rev. Mod. Phys.* 81 (Sep 2009), 1301–1350. Issue 3. https://doi.org/10.1103/RevModPhys.81.1301

[41] SSH Academny. [n. d.]. *Choosing an Algorithm and Key Size.* https://www.ssh.com/academy/ssh/keygen

[42] Gyana Swain. [n. d.]. *Chinese researchers break RSA encryption with a quantum computer.* https://www.csoonline.com/article/3562701/chinese-researchers-break-rsa-encryption-with-a-quantum-computer.html

[43] V8. [n. d.]. *About V8.* https://v8.dev/docs

[44] Efraim Yehuda Weissman, Avraham Merzel, Nadav Katz, and Igal Galili. 2024. Keep it secret, keep it safe: teaching quantum key distribution in high school. 1 (Oktober 2024), 1–23. https://epjquantumtechnology.springeropen.com/articles/10.1140/epjqt/s40507-024-00276-4

[45] xyflow team. [n. d.]. *Overview.* https://reactflow.dev/learn/concepts/terms-and-definitions

A Screenshots der Webanwendung



Abbildung 6: Startseite der Anwendung mit „Beginner Mode“ und „Expert Mode“.

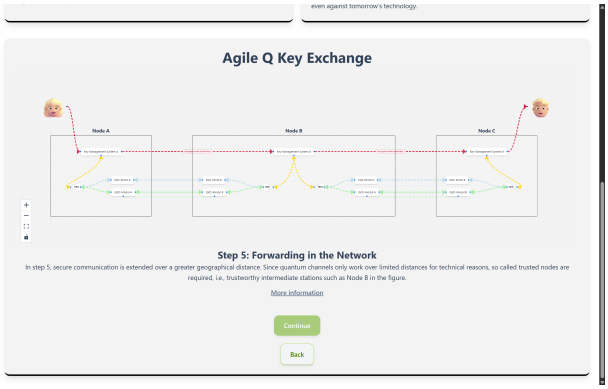


Abbildung 7: Agile Q Key Exchange – Schritt 5: „Forwarding in the Network“ mit Trusted Node (Node B).